Filière : Cyber Sécurité et Intelligence Artificielle (CSIA)

Module : Architecture Logicielle

Rapprot

Nom et Prénom : Abderrazak MOSTAID

Objectif du TP

Déployer une architecture distribuée complète avec 6 composants interconnectés.

Comprendre le rôle de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages.

Implémenter un flux complet : POST commande → stockage MongoDB → cache Redis → notification RabbitMQ.

Tester et observer le comportement distribué avec Postman.

Compétences acquises

A la fin de ce TP, il faut découvrir comment une architecture distribuée fonctionne exactement, plus précisément comment ces parties qui sont séparé communiquent entre eux.

Dans ce TP lorsque on va travailler avec Node.js qui est une runtime environnement pour le langage JS, il faut aussi découvrir comment travailler avec ces outils.

La partie pratique

Pour cette partie, se déroule su le VS code au lieu de Intellij IDEA

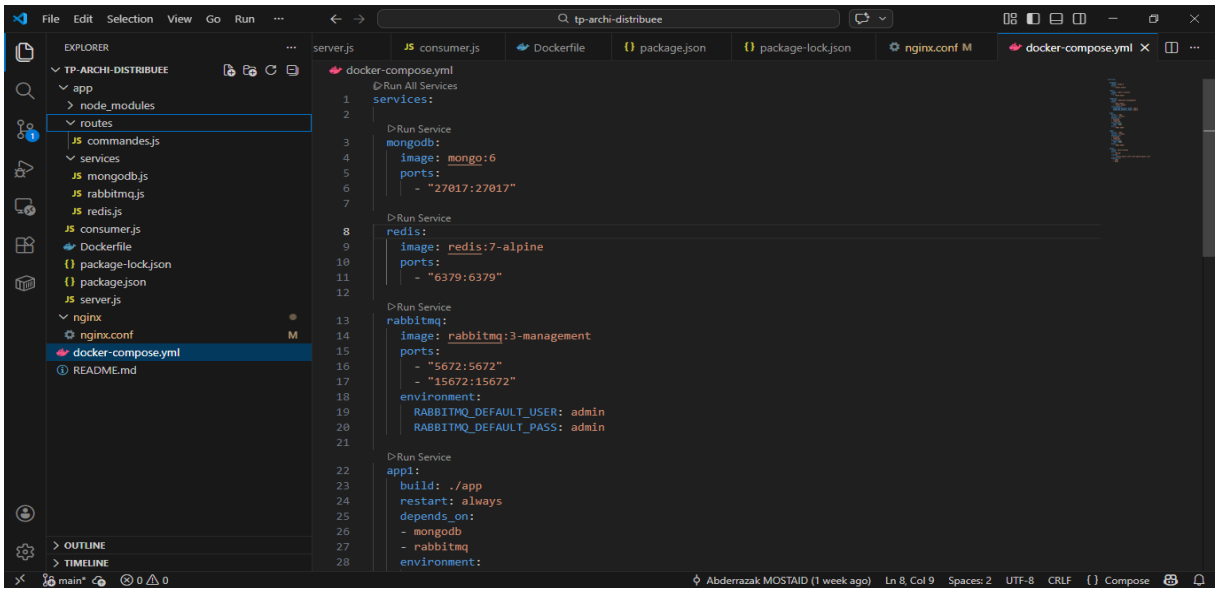
Installation des outils nécessaires

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\PC>node -v
v24.14.0

C:\Users\PC>npm -v
11.9.0

C:\Users\PC>docker -v
Docker version 29.2.0, build 0b9d198

C:\Users\PC>docker compose version
Docker Compose version v5.0.2

C:\Users\PC>_
```

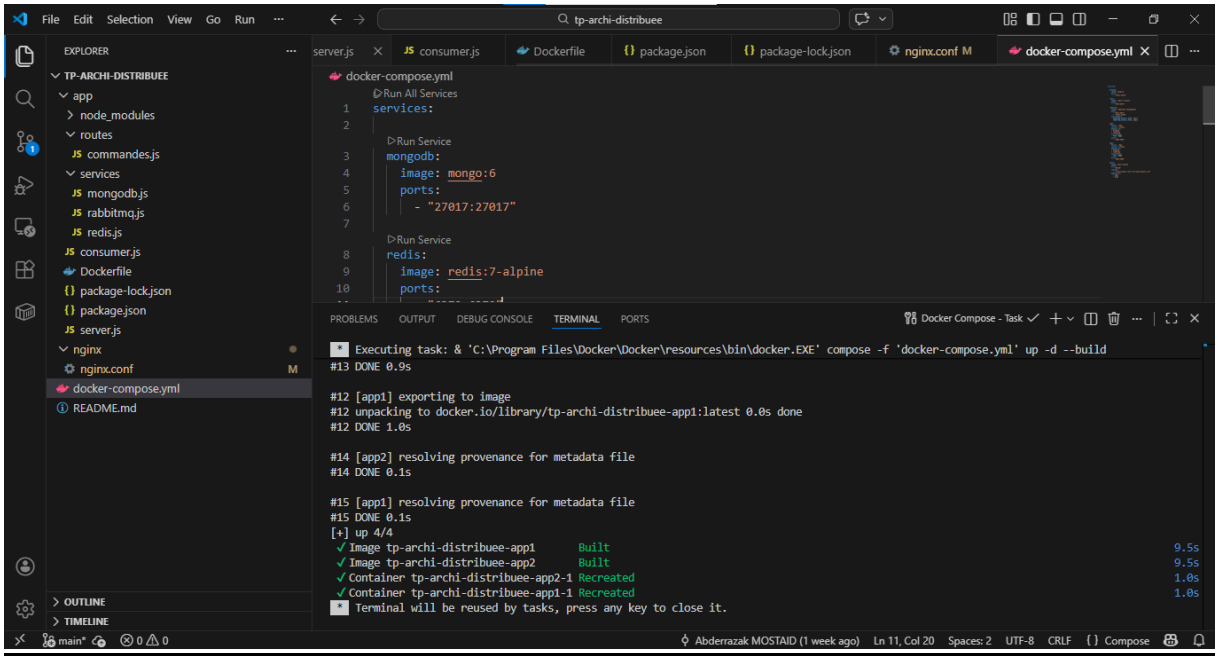
Pulling des Images nécessaires via docker

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\PC\tp-archi-distribuee>docker compose up --build
unable to get image 'mongo:6': error during connect: Get "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.51/images/
mongo:6/json": open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

C:\Users\PC\tp-archi-distribuee>docker compose up --build
[+] up 0/4
 - Image rabbitmq:3-management Pulling                                                    7.8s
 - Image mongo:6               Pulling                                                    7.8s
 - Image redis:7-alpine        Pulling                                                    7.8s
 - Image nginx:alpine          Pulling                                                    7.8s
```

Structure Global du Projet



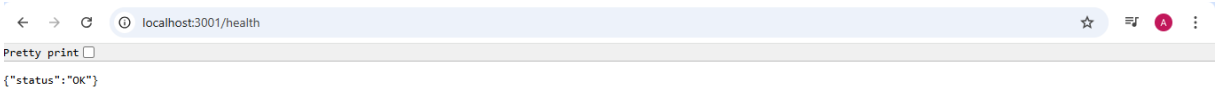
Exécution du code



Lorsque on utilise des images sur docker lors de l'exécution on crée des conteneurs de ces images et on exécute ces conteneurs.

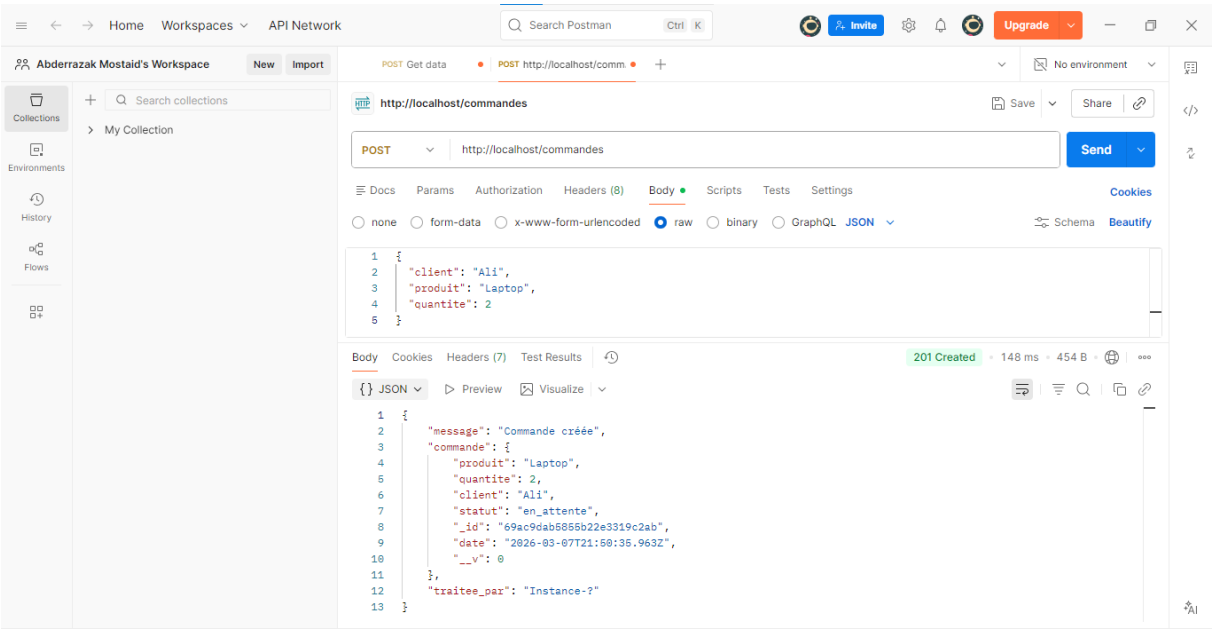
```
C:\Users\PC>docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED        STATUS             PORTS
                                NAMES
47fc8317c649   tp-archi-distribuee-app1   "docker-entrypoint.s…"   2 minutes ago   Up About a minute   0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
                                tp-archi-distribuee-app1-1
18ee461b20fa   tp-archi-distribuee-app2   "docker-entrypoint.s…"   2 minutes ago   Up About a minute   0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
                                tp-archi-distribuee-app2-1
153aa9edffb1   nginx:alpine               "/docker-entrypoint.…"   13 days ago     Up 2 minutes        0.0.0.0:80->80/tcp, [::]:80->80/tcp
                                tp-archi-distribuee-nginx-1
c823b0051e98   rabbitmq:3-management      "docker-entrypoint.s…"   13 days ago     Up 2 minutes        0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672
/tcp, [::]:15672->15672/tcp   tp-archi-distribuee-rabbitmq-1
622d43d3ba89   redis:7-alpine             "docker-entrypoint.s…"   13 days ago     Up 2 minutes        0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
                                tp-archi-distribuee-redis-1
9cf45c1e2fd4   mongo:6                    "docker-entrypoint.s…"   13 days ago     Up 2 minutes        0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
                                tp-archi-distribuee-mongodb-1
```

Health check

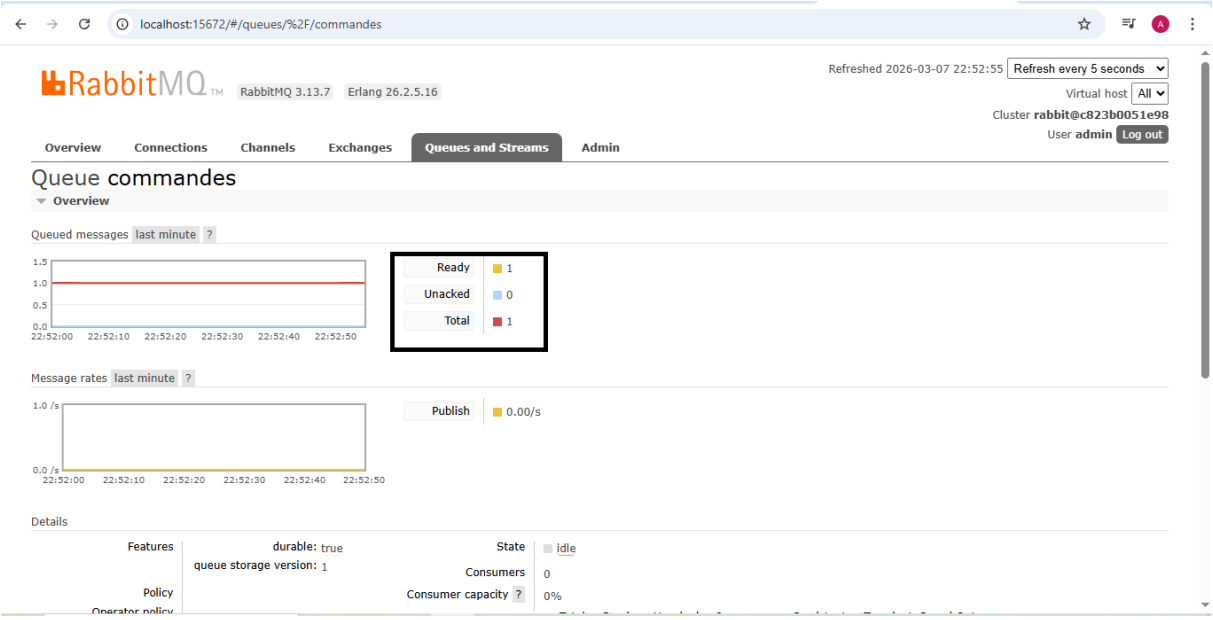


Maintenant on va tester la connectivité entre les différents services, on le faire avec Postaman, ou on va créer des requêtes http pour le getting et le posting.

Vérification avec POST requests

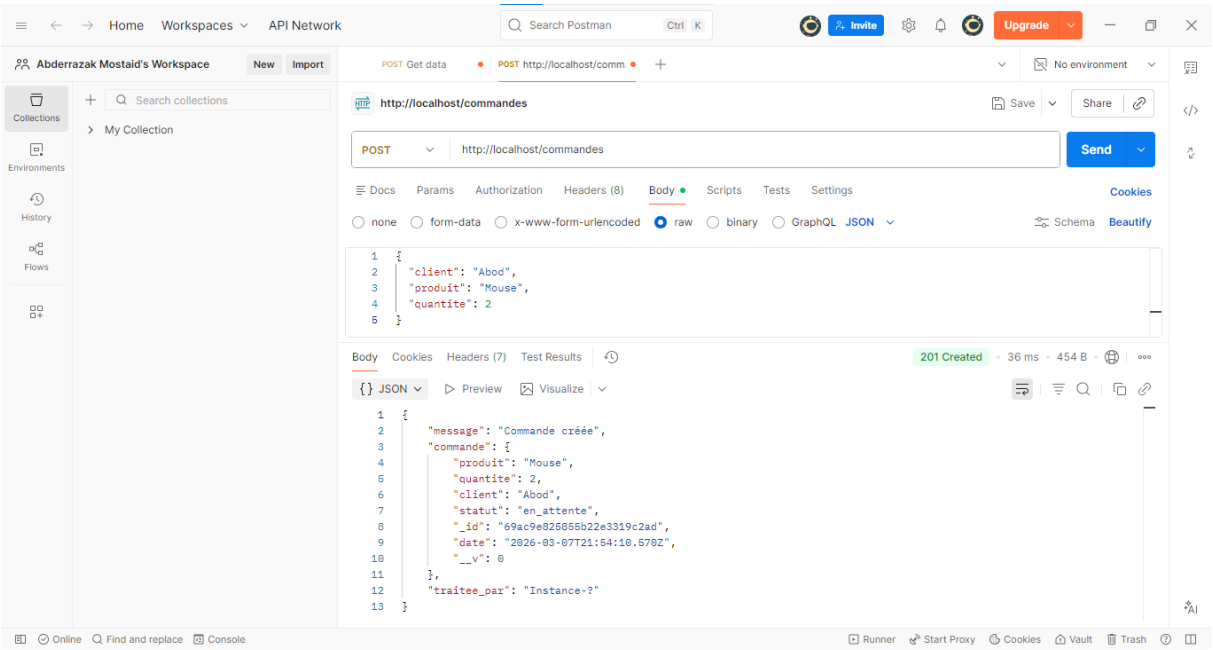


RabbitMQ response

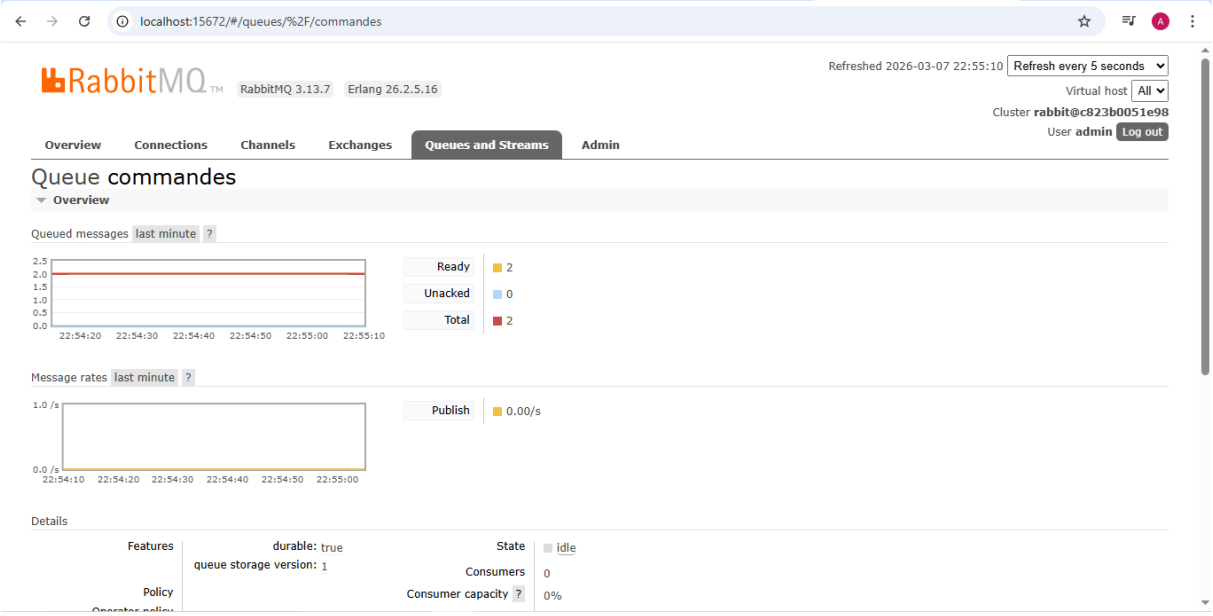


On va faire une deuxième vérification pour voir la connectivité ou bien le lien entre Postman et RabbitQM.

Vérification avec POST requests (2)



RabbitMQ response



La vérification des données dans la Base de données (Cache)

```
commandesDB> show dbs
admin      40.00 KiB
boutique   72.00 KiB
config    108.00 KiB
local      72.00 KiB
commandesDB> use boutique
switched to db boutique
boutique> show collections
commandes
boutique> db.commandes.find().pretty()
[
  {
    _id: ObjectId('69ac9dab5855b22e3319c2ab'),
    produit: 'Laptop',
    quantite: 2,
    client: 'Ali',
    statut: 'en_attente',
    date: ISODate('2026-03-07T21:50:35.963Z'),
    __v: 0
  },
  {
    _id: ObjectId('69ac9e825855b22e3319c2ad'),
    produit: 'Mouse',
    quantite: 2,
    client: 'Abod',
    statut: 'en_attente',
    date: ISODate('2026-03-07T21:54:10.570Z'),
    __v: 0
  }
]
boutique>
```

Alors maintenant il est clair que tous les composants sont fonctionnels et ils sont interconnecté entre eux, ce qui montre que notre architecture distribuée a été construit avec succès.